



Vietnam National University – HCMC, International University

FINAL REPORT GROUP 5

Topic 19: Online voting system database

Course: Principles of Database Management (IT079IU)

Instructor: Assoc. Prof. Dr. Nguyễn Thị Thúy Loan

Student ID	Name	Role
ITITWE24075	Nguyễn Khang Vỹ	Leader
ITITWE24002	Nguyễn Ngọc Quang Anh	Member
ITITWE24030	Phạm Nguyễn Phúc An	Member
ITITWE24068	Mai Trần Tâm	Member
ITITWE24015	Nguyễn Minh Nhân	Member
ITITWE24044	Nguyễn Võ Minh Huy	Member
ITITWE24012	Phan Nhật Huy	Member
ITITWE24069	Nguyễn Xuân Thành	Member

ABSTRACT

The Online Voting System Database is designed to provide a secure, efficient, and transparent digital voting platform that replaces traditional manual voting methods. The system enables administrators to manage elections, register voters, maintain candidate information, and generate voting results, while voters can securely access the system to view available elections and cast their votes following the one voter–one vote principle.

The database is developed using a relational database model implemented in MySQL. It focuses on maintaining data integrity, security, and reliability through the use of constraints, authentication mechanisms, and validation rules. These mechanisms help protect sensitive voter information and prevent unauthorized or duplicate voting.

This report presents the database design of the Online Voting System, including the Entity–Relationship Diagram (ERD) and the relational schema that defines the structure of the database. The design demonstrates how database management concepts can be applied to develop a practical and scalable digital voting system.

TABLE OF CONTENTS

CHAPTER 1	: INTRODUCTION	1
1.1	Background	1
1.2	Project Objectives	1
CHAPTER 2	: PROJECT ORGANIZATION	2
2.1	Tools and Software Used Table	2
2.2	Task Allocation Table	3
CHAPTER 3	: METHODOLOGY	4
3.1	Database Design	4
3.1.1	Entity-Relationship Diagram (ERD)	5
3.1.2	Relational Schema	6
3.1.3	Table Creation and Normalization (3NF Analysis)	8
3.1.4	Data Collection and Insertion.	10
3.1.5	Database Queries	11
3.2	Application/Website structure	13
3.2.1	Project Architecture	13

3.2.2	Class Structure.....	14
3.2.3	Database Connection Implementation	17
3.2.4	Graphical User Interface (GUI) Design	18
3.2.5	Layout and Functionality	18
CHAPTER 4	DISCUSSION & IMPLEMENTATION.....	27
4.1	Implementation Process.....	28
4.1.1	Database Setup and Schema Instantiation	28
4.1.2	Data Access Layer	28
4.1.3	Entity Layer	28
4.1.4	GUI Implementation	29
4.2	Results & Analysis.....	29
4.2.1	Voter Registration and Authentication	29
4.2.2	Election Participation.....	29
4.2.3	Results Transparency	29
4.2.4	Audit Trail	30
4.3	Challenges & Solutions	30
CHAPTER 5	CONCLUSION.....	31
5.1	Summary.....	31
5.2	Reflection	32
5.3	Implications	32
5.4	Future Work.....	33
CHAPTER 6	: REFERENCES	34

List of Figures

Figure 3.1.1.	Entity–Relationship Diagram of the Online Voting System Database.....	6
Figure 3.1.2.	Relational Schema of the Online Voting System Database.....	8
Figure 3.2.1.	Three-Layer Application Architecture.....	13
Figure 3.2.2.	UML DIAGRAM	16

List of Tables

Table 2.1.	Tools and Software Used	2
Table 2.2	Task Allocation.....	3

MAIN CONTENT

CHAPTER 1 : INTRODUCTION

1.1 Background

Traditional voting systems often rely on paper ballots and manual vote counting, which can be time-consuming and prone to human error. Managing voter registration, candidate information, and election results manually also increases the risk of data inconsistency and inefficiency. As technology continues to develop, digital solutions have become an effective approach to improve the efficiency, transparency, and reliability of voting processes.

An Online Voting System provides a platform where elections can be organized and managed electronically. Through a well-designed database system, important information such as voter details, candidate data, election schedules, and voting records can be stored and managed efficiently. A structured database ensures data integrity, prevents duplicate voting, and allows administrators to monitor the election process more effectively.

Therefore, designing a reliable database is a critical step in developing a secure online voting platform. This project focuses on building a relational database structure that supports the main functions of an online voting system, including voter management, election management, candidate registration, and vote recording.

1.2 Project Objectives

The main objective of the Online Voting System Database project is to design and implement a reliable and well-structured database system that supports a secure and transparent online voting process. The project aims to apply fundamental principles of database management, including data modeling, normalization, and relational integrity.

Specifically, the objectives of the project are:

- To analyze the functional requirements of an online voting system and identify key entities such as voters, elections, candidates, and votes.
- To design an Entity-Relationship Diagram (ERD) and convert it into a relational database schema using appropriate primary and foreign keys.
- To implement the database using MySQL and ensure data consistency through constraints, relationships, and validation rules.
- To perform data retrieval and analysis using SQL queries and relational algebra expressions.
- To integrate the database with a basic Java-based user interface for demonstrating voting operations.
- To enhance understanding of real-world database applications by developing a practical and socially relevant system.

CHAPTER 2 : PROJECT ORGANIZATION

2.1 Tools and Software Used Table

Table 2.1.Tools and Software Used

Task	Tool/Software
ERD Design	Draw.io
Database Implementation	MySQL
Relational Algebra Trees	Draw.io
Report Writing	MS Word
Java Interface	Java (NetBeans), Visual Studio Code
Presentation	PowerPoint, Canva
Reference Citation	Zotero
Generate Data	Mockaroo+ChatGPT+Claude

2.2 Task Allocation Table

Table 2.2 Task Allocation

Time	No.	Tasks	Members
Week 1	1	Getting an overview of databases, SQL, and MySQL from BB slides, Books and YouTube.	All
Week 2 Making a proposal	1	Build a proposal framework and assign tasks	Vỹ
	2	Meeting	All
	2.1	Brief explanation of the topic	Nhân, Anh
	2.2	The project's objectives	Nhật Huy
	2.3	List tools and programs will be used	Vỹ
Week 3 Collect data resources and form a basic database system	1	Do research, find data sources – raw data	Thành, Minh Huy
	2	Filter information for ERD	Vỹ, Nhật Huy
	3	Design an ERD and a Schema Diagram	Minh Huy, Tâm
	4	Relational Algebra	Nhân, Tâm
	5	Data Retrieval with Relational Algebra and Tree	Tâm
Week 4-7 Apply SQL and Java	1	Build a midterm report framework and assign tasks	Vỹ
	2	Making UML	Vỹ
	3	Use MySQL with the data from synthesized information	Nhật Huy, Tâm, Vỹ, Anh

	4	Apply java code	Nhật Huy, Thành, Tâm
	5	Running code and debug	Vỹ
	6	Meeting	All
Week 8 Final report	1	Build a final report framework and assign tasks	Vỹ
	2	Introduction	Nhân, Anh
	3	Task Timeline & Division	Nhật Huy
	4	Methodology	Anh, Thành
	5	Conclusion & Future Work	All
Week 9 Preparing for Presentation	1	Introduction	Vỹ
	2	ERD Design, Relational Model, Relational Algebra Queries	Nhân, Tâm, Minh Huy
	3	SQL Implementation	Nhật Huy
	4	Normalization	Anh, Thành
	5	Java User Interface	Nhật Huy, Thành, Tâm, An
	6	Conclusion	Vỹ

CHAPTER 3 : METHODOLOGY

3.1 Database Design

The database system is designed to support an electronic voting (e-voting) platform, ensuring requirements related to:

- Data integrity

- Security and auditability
- Traceability of voting activities
- Verification and transparency of election results

The database is implemented using the relational model and is normalized up to the Third Normal Form (3NF) to reduce redundancy and avoid data anomalies.

3.1.1 Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram captures all entities, their attributes, and the associations between them. Nine entities were identified during the requirements analysis phase:

- VOTER – represents registered citizens who participate in elections.
- ELECTION – an electoral event with a defined period and status lifecycle.
- CANDIDATE – a participant contending in a specific election.
- BALLOT – the official voting form associated with an election.
- TOKEN – a single-use authorisation credential issued per voter per election.
- VOTE – the immutable encrypted record of a submitted ballot.
- ELECTION_RESULT – the certified aggregate of votes per candidate.
- AUDIT_LOG – a timestamped record of every voter action in the system.
- SECURITY_EVENT – a record of anomalous or security-relevant system events.

In addition, the associative entity VOTER_ELECTION resolves the many-to-many relationship between voters and elections, storing eligibility status and the has_voted flag.

Key Relationships:

- A VOTER registers for many ELECTIONS through VOTER_ELECTION (many-to-many).
- An ELECTION contains many CANDIDATES, BALLOTS, TOKENS, VOTES, and ELECTION_RESULTS (one-to-many).
- A TOKEN is linked to exactly one BALLOT and one ELECTION, and may be used at most once.

- A VOTE references one CANDIDATE, one ELECTION, one BALLOT, and one TOKEN, ensuring full traceability.
- AUDIT_LOG and SECURITY_EVENT records reference the VOTER who triggered them, supporting forensic investigation.

The overall database structure of the system is illustrated in Figure 3.2.1:

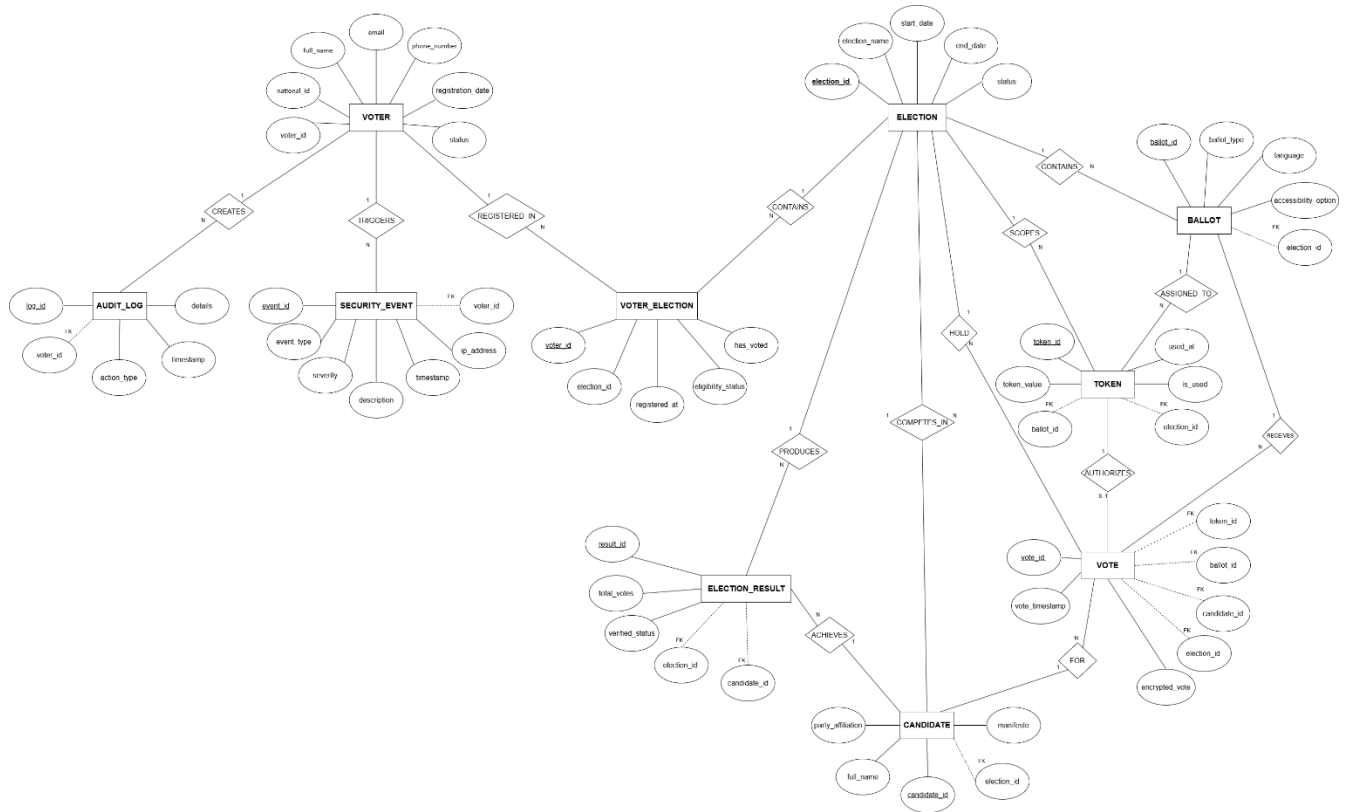


Figure 3.1.1.Entity–Relationship Diagram of the Online Voting System Database

3.1.2 Relational Schema

Relation	Primary Key	Foreign Keys	Notable Attributes
VOTER	voter_id	—	national_id, full_name, email, phone_number, registration_date, status
ELECTION	election_id	—	election_name, start_date, end_date, status

Relation	Primary Key	Foreign Keys	Notable Attributes
CANDIDATE	candidate_id	election_id	full_name, party_affiliation, manifesto
BALLOT	ballot_id	election_id	ballot_type, language, accessibility_option
TOKEN	token_id	election_id, ballot_id	token_value, is_used, used_at
VOTE	vote_id	candidate_id, election_id, ballot_id, token_id	encrypted_vote, vote_timestamp
ELECTION_RESULT	result_id	election_id, candidate_id	total_votes, verified_status
AUDIT_LOG	log_id	voter_id	action_type, timestamp, details
SECURITY_EVENT	event_id	voter_id	event_type, severity, description, timestamp, ip_address
VOTER_ELECTION	voter_id + election_id	voter_id, election_id	eligibility_status, has_voted, registered_at

The overall database structure of the system is illustrated in Figure 3.1.2:

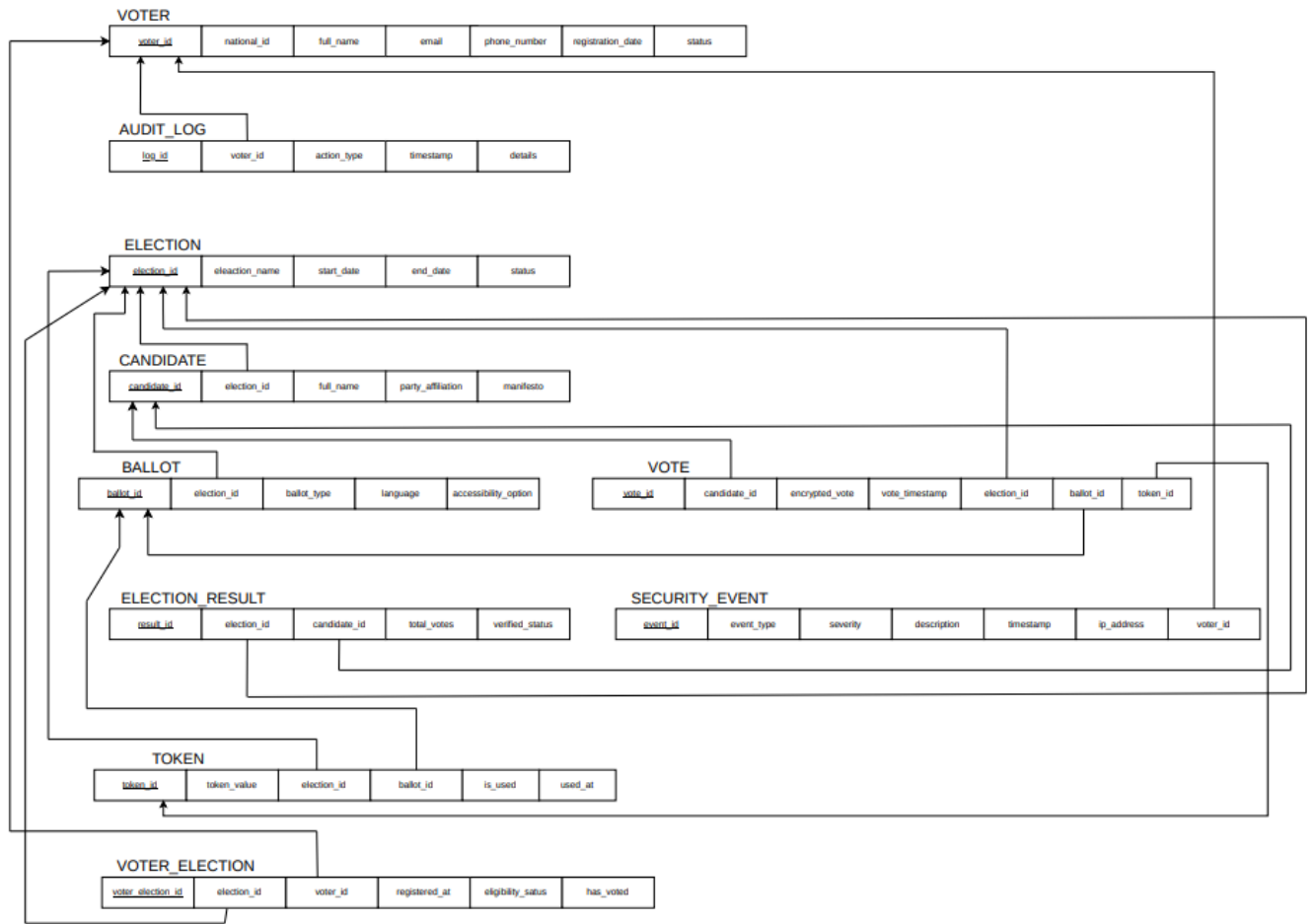


Figure 3.1.2. Relational Schema of the Online Voting System Database

3.1.3 Table Creation and Normalization (3NF Analysis)

All relations were normalised to at least the Third Normal Form (3NF) to eliminate redundancy and prevent update anomalies. The normalisation process proceeded as follows:

Table	Primary Key	1NF (Atomic, no repeating)	2NF (No partial dependency)	3NF (No non-key $\rightarrow$ non-key)
VOTER	voter_id	All attributes (name, email, phone, etc.) are single-valued; no repeating groups	Single PK	All attributes depend only on voter_id; no attribute depends on another non-key

AUDIT_LOG	log_id	Each field (action_type, timestamp, details) is atomic	Single PK	All attributes depend only on log_id; no non-key dependency
ELECTION	election_id	Attributes (name, dates, status) are atomic	Single PK	No non-key attribute determines another
CANDIDATE	candidate_id	All fields are atomic (name, party, manifesto)	Single PK	All attributes depend only on candidate_id
BALLOT	ballot_id	ballot_type, language, accessibility_option are atomic	Single PK	No non-key → non-key dependency
VOTE	vote_id	Each attribute (candidate_id, ballot_id, token_id, etc.) is atomic	Single PK	All attributes depend on vote_id only; no internal non-key dependency
ELECTION_RESULT	result_id	total_votes, verified_status are atomic values	Single PK	No attribute depends on another non-key
SECURITY_EVENT	event_id	event_type, severity, description, etc. are atomic	Single PK	All attributes depend only on event_id
TOKEN	token_id	token_value, is_used, used_at are atomic	Single PK	No non-key → non-key dependency

VOTER_ ELECTION	(voter_id, election _id)	All attributes are atomic	All attributes depend on full composite key (not just voter_id or election_id)	No non-key attribute determines another
--------------------	--------------------------------	---------------------------	--	--

The database schema is designed in Third Normal Form (3NF). All relations satisfy 1NF, 2NF, and 3NF because all attributes are atomic, no partial dependencies exist, and no transitive dependencies are present within each relation.

3.1.4 Data Collection and Insertion.

The dataset used in this project is generated mainly using Mockaroo, a synthetic data generation tool that allows the creation of realistic test data based on custom schemas.

Mockaroo is used to simulate real-world e-voting data for testing database design, constraints, and query performance.

1. Parent tables generated first

These tables do not depend on foreign keys:

- VOTER
- ELECTION
- CANDIDATE (after ELECTION)
- BALLOT (after ELECTION)
- TOKEN (after ELECTION + BALLOT)

=> These are generated first to ensure referential integrity.

2. Dependent / transactional tables generated later

- VOTE (depends on candidate_id, election_id, token_id, ballot_id)
- AUDIT_LOG (depends on voter_id)
- SECURITY_EVENT (depends on voter_id, optional vote/token context)
- VOTER_ELECTION (depends on voter_id + election_id)

Since Mockaroo does not automatically enforce relational constraints, FK consistency is ensured using:

Method: Export primary key lists first

- Export voter_id from VOTER table
- Export election_id from ELECTION table
- Use them as “Custom List” fields in Mockaroo

3.1.5 Database Queries

Three complementary query frameworks were employed to retrieve and analyse data:

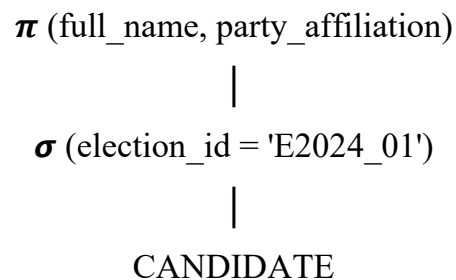
Example 1:

Retrieve the Full Name (full_name) and Party Affiliation (party_affiliation) of all candidates participating in the election with the ID 'E2024_01'

- Relational Algebra Expression:

$\pi_{\text{full_name, party_affiliation}} (\sigma_{\text{election_id}='E2024_01'}(\text{CANDIDATE}))$

- Trees:



- SQL queries:

```
SELECT c.full_name, c.party_affiliation
FROM candidate
WHERE c.election_id = 'E2024_01';
```

- Explanation:

This query retrieves the full name and party affiliation of all candidates who are participating in a specific election identified by the ID 'E2024_01'. The relational algebra expression first applies a selection (σ) operation to filter tuples in the CANDIDATE table where the election_id matches the given value. Then, a projection (π) operation is used to extract only the required attributes, namely full_name and party_affiliation. The query tree reflects this execution order, where the selection is performed before projection to reduce unnecessary data processing. The SQL query implements the same logic using a WHERE clause to filter records and returns only

the specified columns. This query is useful for displaying candidate information for a particular election.

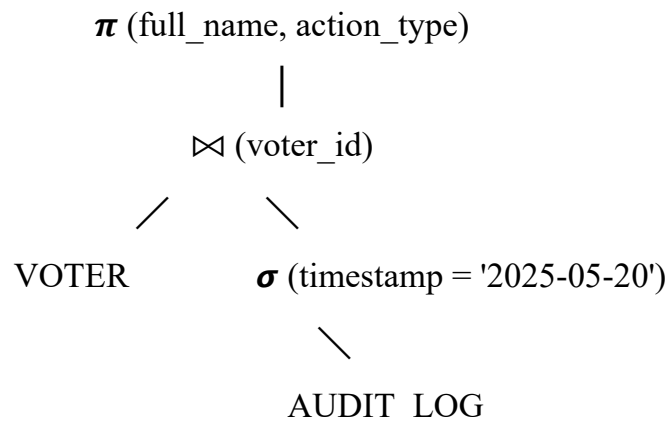
Example 2:

Retrieve the Voter Name (full_name) and Action Type (action_type) for all audit logs recorded on the date '2025-05-20'

- Relational Algebra Expression:

$\pi_{\text{full_name, action_type}}(\text{VOTER} \bowtie_{\text{VOTER.voter_id=AUDIT_LOG.voter_id}} (\sigma_{\text{timestamp='2025-05-20'}}(\text{AUDIT_LOG})))$

- Trees:



- SQL queries:

```

SELECT v.full_name, a.action_type
FROM AUDIT_LOG a
JOIN VOTER v ON a.voter_id = v.voter_id
WHERE DATE(a.timestamp) = '2025-05-20';
  
```

- Explanation:

This query retrieves the full name of voters and the corresponding action types recorded in the audit logs on a specific date ('2025-05-20'). In relational algebra, the process begins with a selection (σ) operation on the AUDIT_LOG table to filter records by the given date. Then, a join ($\bowtie$) operation is performed between AUDIT_LOG and VOTER using the common attribute voter_id to associate each log entry with the corresponding voter. Finally, a projection (π) operation extracts the required attributes: full_name and action_type. The query tree illustrates this sequence by first filtering the audit logs, then joining with the VOTER table, and finally projecting the desired fields. The SQL query follows the same logic by using a JOIN

clause to combine both tables and a WHERE condition to filter records by date. This query is useful for tracking voter activities on a specific day for auditing and monitoring purposes.

3.2 Application/Website structure

3.2.1 Project Architecture

Layer	Technology	Responsibility
Presentation Layer	Java Swing (GUI)	User interface – forms, buttons, dialogs, and event handling
Business Logic Layer	Plain Java Classes	Domain rules – vote validation, token management, eligibility checks
Data Access Layer	JDBC + MySQL	SQL queries, CRUD operations, and database connection management

Figure 3.2.1 below illustrates the high-level system structure:

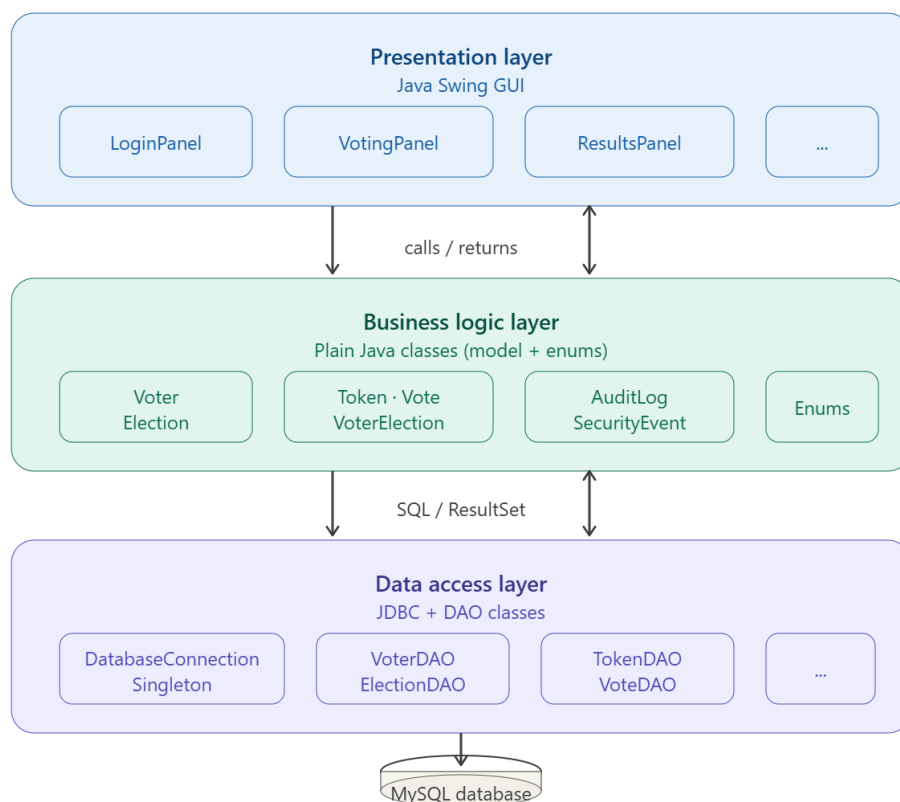


Figure 3.2.1. Three-Layer Application Architecture

The three layers communicate strictly top-down: the Presentation Layer calls the Business Logic Layer, which in turn calls the Data Access Layer. No layer skips another. This ensures that all validation rules are enforced consistently, regardless of which GUI screen triggers an action.

3.2.2 Class Structure

The system is organised into four Java packages, each with a clearly defined role:

`com.evoting.model:`

- Contains all domain classes representing the core data of the system.
- These classes correspond to database tables but are implemented as plain Java objects (POJOs) without using JPA annotations.
- Each class includes fields, constructors, getters, and setters.
- BaseEntity (abstract)
 - Provides common attributes such as `createdAt` and `updatedAt` for reuse across entities.
- Core entities
 - Voter, Election, Candidate, VoterElection, Token, Vote, ElectionResult
 - Represent the main business objects in the voting system.
- Security entities
 - AuditLog, SecurityEvent
 - Used for tracking user activities and detecting suspicious behavior.
- Ballot (abstract class)
 - Defines common behavior such as `render()` and `getAccessibilityOptions()`
 - StandardBallot provides a concrete implementation.

`com.evoting.enums:`

- Defines enumerations to restrict valid values and improve data consistency.
- VoterStatus → PENDING, ACTIVE, SUSPENDED, DEACTIVATED

- ElectionStatus → UPCOMING, ACTIVE, CLOSED, VERIFIED
- Severity → LOW, MEDIUM, HIGH, CRITICAL
- ActionType → represents system actions such as REGISTER, LOGIN, VOTE, TOKEN_ISSUED, etc.

com.evoting.dao:

- Implements the Data Access Object (DAO) pattern.
- Each entity has a corresponding DAO class responsible for interacting with the database using JDBC.
- Key characteristics:
 - Each DAO receives a java.sql.Connection
 - Encapsulates all SQL operations (INSERT, SELECT, UPDATE, DELETE)
 - Provides methods such as:
 - findById()
 - save()
 - delete()
- Example:
 - VoterDAO includes additional methods like:
 - findByNationalId()
 - findByEmail()

com.evoting.ui:

- Contains all user interface components built using Java Swing.
- Main components:
 - LoginPanel
 - RegisterPanel
 - ElectionListPanel
 - VotingPanel

- ResultsPanel
 - AdminPanel
- Each component:
 - Extends JPanel
 - Handles specific user interactions
- The main window (JFrame) uses CardLayout to switch between panels dynamically.

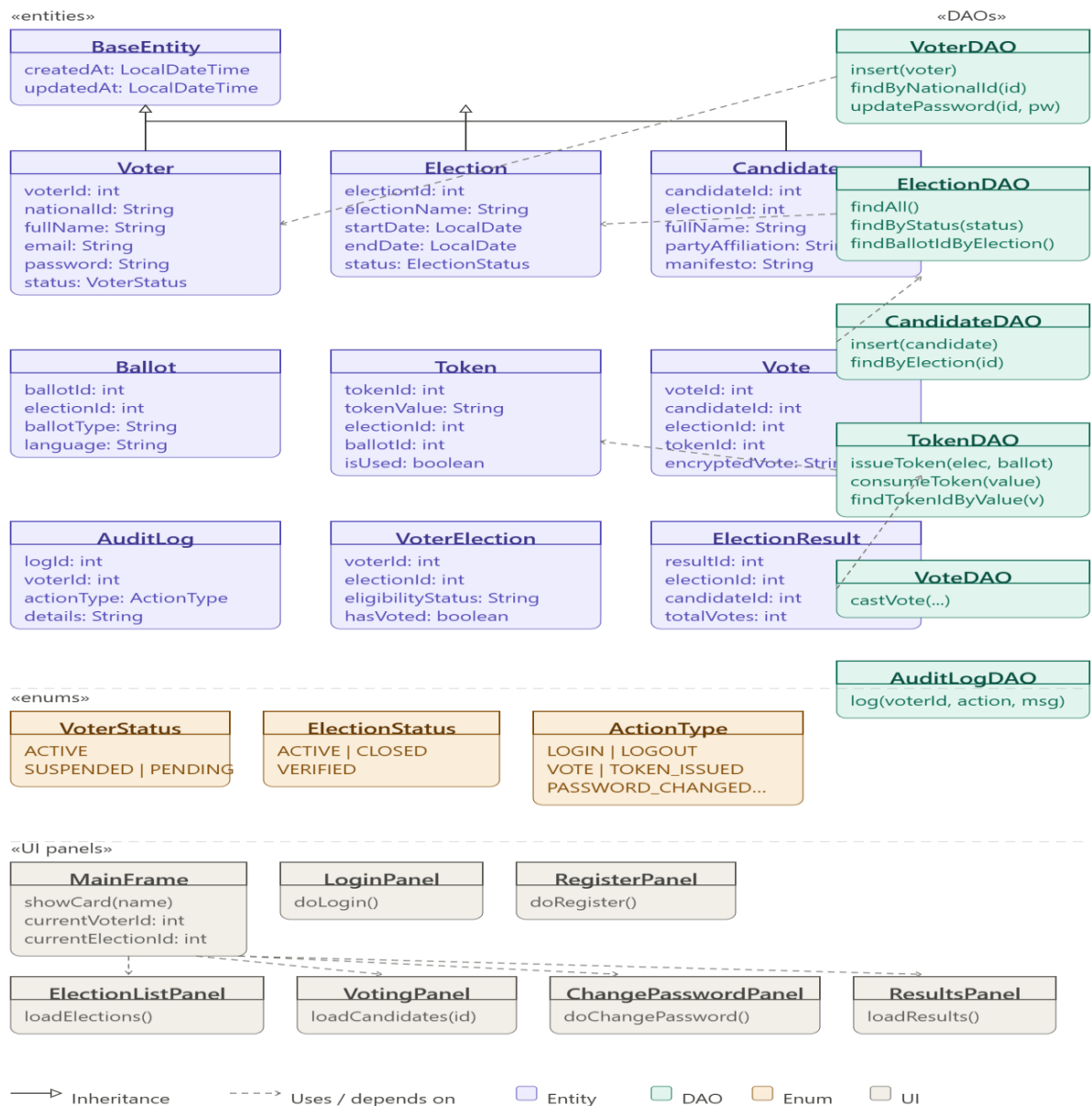


Figure 3.2.2.UML DIAGRAM

3.2.3 Database Connection Implementation

The graphical user interface (GUI) is implemented using Java Swing.

The main application window is a JFrame that uses a CardLayout to switch between different panels without opening new windows. This provides a single-window navigation experience similar to a wizard-style interface.

Main Screens:

The system includes the following screens:

Screen / Panel	Class Name	Purpose
Login Screen	LoginPanel	Authenticates voters using National ID and password
Registration Screen	RegisterPanel	Allows new voters to register
Election List	ElectionListPanel	Displays available elections
Voting Screen	VotingPanel	Displays ballot and allows vote submission
Results Screen	ResultsPanel	Displays vote results per candidate for a selected election

Button and Form Behaviour:

All interactive components use ActionListener to trigger backend logic through DAO classes.

- Login Button
 - Validates that all fields are filled
 - Calls VoterDAO.findByNameNationalId()
 - Verifies credentials
 - Ensures VoterStatus == ACTIVE before granting access
- Vote Submit Button
 - Disabled until a candidate is selected (radio button)
 - On click:
 - Calls TokenDAO.consumeToken() to mark the token as used
 - Inserts a new record into the VOTE table
 - Prevents duplicate voting via token validation
- Register Button
 - Checks for duplicate National ID and email using VoterDAO
 - Inserts a new Voter with status PENDING
 - Requires admin approval to become ACTIVE
- Confirmation Dialog
 - Critical actions (e.g., vote submission) require confirmation using:

JOptionPane.showConfirmDialog()

- Prevents accidental operations

Input Validation:

Client-side validation is performed before executing any SQL operations:

- Required Fields

Empty fields trigger error messages displayed via JLabel

- Email Format

Validated using a regular expression in RegisterPanel

- Password Policy

Minimum length of 6 characters enforced

- Election Status Check

Only elections with status ACTIVE can be voted on

Others display a warning dialog

- Candidate Selection

The "Submit Vote" button remains disabled until a candidate is selected

3.2.4 Graphical User Interface (GUI) Design

The E-Voting System employs Java Swing as its GUI framework. The interface is built around a single window (MainFrame) that uses a CardLayout manager to swap between six distinct panels without opening additional windows. This design keeps the application lightweight and navigation predictable for the voter. The window is fixed at 800 × 600 pixels and centered on the screen at startup.

3.2.5 Layout and Functionality

The application is structured as a card-based single-window interface. MainFrame acts as the container and router, holding all six panels simultaneously; only one card is visible at a time. Navigation is driven exclusively by button events — no menus or tab bars are exposed to the user. The CardLayout ensures that switching panels is instantaneous and that each panel is initialized only once, reducing overhead.

The six screens and their navigation flow are described below:

Screen (Card)	Class	Purpose	Navigates To

Login	LoginPanel	Voter authentication entry point	Elections List, Register
Register	RegisterPanel	New voter self-registration form	Login
Elections List	ElectionListPanel	Table of all available elections	Voting, Results, Change Password, Login
Voting	VotingPanel	Candidate selection and ballot submission	Elections List
Results	ResultsPanel	Vote count retrieval by election	Elections List
Change Password	ChangePasswordPanel	In-session password update form	Elections List

Login Panel

The Login Panel (LoginPanel.java) is the first screen presented when the application launches. It uses a GridBagLayout to center its form elements both horizontally and vertically within the 800×600 window. The panel contains the following elements:

Component	Description
Title Label	Bold, 22pt sans-serif heading reading 'E-Voting System', spanning both columns and centered.
National ID Field	A JTextField (20 columns wide) for entering the voter's national identifier.
Password Field	A JPasswordField (20 columns wide) that masks input characters.

Error Label	A red-colored JLabel initially set to a blank space. Displays inline validation messages such as 'National ID not found' or 'Incorrect password' without requiring a dialog popup.
Login Button	Triggers the doLogin() method. Disabled during the asynchronous database lookup (SwingWorker) to prevent double-submissions.
Register Button	Navigates to the Register Panel via <code>frame.showCard(CARD_REGISTER)</code> .

Authentication is performed asynchronously using a SwingWorker to avoid blocking the Event Dispatch Thread. The Login button is re-enabled in the done() callback regardless of the outcome. Pressing Enter inside the password field also triggers login, improving keyboard usability.

Register Panel

The Register Panel (RegisterPanel.java) provides new voters with a self-service registration form. Layout is GridBagLayout, consistent with the Login Panel. It contains:

Component	Description
Title Label	Bold, 18pt heading 'Register New Voter'.
National ID Field	JTextField for the voter's national ID number.
Full Name Field	JTextField for the voter's full legal name.
Email Field	JTextField with client-side regex validation (<code>^[\\w.-]+@[\\w.-]+\\. [a-zA-Z]{2,}\$</code>).
Password Field	JPasswordField; enforces a minimum length of 6 characters before submission.

Message Label	A JLabel that alternates between red (error) and green (success) foreground colors to indicate the registration outcome.
Register Button	Submits the form via a SwingWorker; disabled during the DB operation to prevent duplicate submissions.
Back to Login	Returns the user to the Login Panel without clearing any fields, allowing the voter to review their entries.

On successful submission the account is created with a PENDING status, and the voter receives the message 'Registration submitted. Awaiting admin approval.' All form fields are then cleared. Duplicate national ID or email addresses are detected by VoterDAO before the insert is attempted and surfaced through the message label.

Elections List Panel

The Elections List Panel (ElectionListPanel.java) is the main hub the voter sees after logging in. It uses a BorderLayout. The key components are:

Component	Description
Title Label (NORTH)	18pt bold heading 'Available Elections'.
User Label (NORTH-EAST)	Displays 'Logged in' to confirm session state.
JTable (CENTER)	Non-editable table with five columns: ID, Name, Start Date, End Date, and Status. Row height is 28px. Single-selection mode is enforced so only one election can be acted upon at a time.
Status Label (SOUTH-WEST)	Displays 'Loading elections...' during the SwingWorker fetch and then shows the election count (e.g., '3 election(s) found.') once complete.

Vote Button (SOUTH-EAST)	Enabled only after data is loaded. Validates that the selected election has ACTIVE status before navigating to the Voting Panel. Shows a JOptionPane warning if no row is selected or if the election is not active.
View Results Button	Navigates to the Results Panel.
Change Password Button	Navigates to the Change Password Panel.
Logout Button	Resets currentVoterId and currentElectionId to -1 and returns to the Login Panel.

Data is refreshed automatically every time this panel is shown, via the loadElections() call inside MainFrame.showCard(). This prevents stale data from appearing if an election's status changes during a session.

Voting Panel

The Voting Panel (VotingPanel.java) allows a logged-in voter to select a candidate and cast a ballot. Its layout is BorderLayout. Components include:

Component	Description
Header (NORTH)	A two-row header: a bold 18pt 'Cast Your Vote' title above a dynamic label that displays the name of the currently loaded election.
Candidate Panel (CENTER)	A scrollable JScrollPane wrapping a BoxLayout panel. Each candidate is rendered as a 'card' containing: a JRadioButton labeled with the candidate's full name and party affiliation, and an italicized HTML label showing the candidate's manifesto text (if present). Cards are separated by JSeparator lines.

ButtonGroup	Ensures that only one candidate radio button can be selected at a time across all dynamically generated buttons.
Submit Vote Button (SOUTH)	Disabled until a candidate is selected via the radio button group. Triggers a JOptionPane confirmation dialog ('Confirm your vote? This cannot be undone.') before initiating the vote submission SwingWorker.
Back Button (SOUTH)	Returns to the Elections List Panel without submitting a vote.
Status Label (SOUTH)	Displays errors in red, such as 'Please select a candidate.' or 'Vote failed: Token already used.'

Candidate data is loaded asynchronously by `loadCandidates(electionId)`, which is called by `MainFrame` before the card becomes visible. The ballot ID is also fetched from the database at load time (via `ElectionDAO.findBallotIdByElection()`) and stored in the panel's `currentBallotId` field for use during vote submission. On successful vote, a confirmation dialog is shown and the voter is automatically returned to the Elections List Panel.

Results Panel

The Results Panel (`ResultsPanel.java`) allows any authenticated voter to query the vote tallies for a specific election. Its layout is `BorderLayout`. Components are:

Component	Description
Title Label (NORTH)	18pt bold 'Election Results' heading.
Search Bar (NORTH sub-panel)	A <code>FlowLayout</code> row containing: a 'Election ID:' label, an 8-column <code>JTextField</code> for the ID input, a 'Load Results' button, and a 'Back' button.

JTable (CENTER)	Non-editable table with three columns: Candidate Name (200px preferred), Party (160px preferred), and Total Votes (100px preferred). Row height is 28px.
Total Votes Label (SOUTH)	Displays the aggregate vote count across all candidates once results are loaded (e.g., 'Total votes: 47').

Input validation ensures the election ID field is non-empty and numeric before querying the database. Results are loaded asynchronously. For each candidate ID in the result map, the panel performs a secondary lookup via `CandidateDAO.findById()` to resolve the candidate's full name and party affiliation, providing a human-readable display instead of raw integer keys.

Change Password Panel

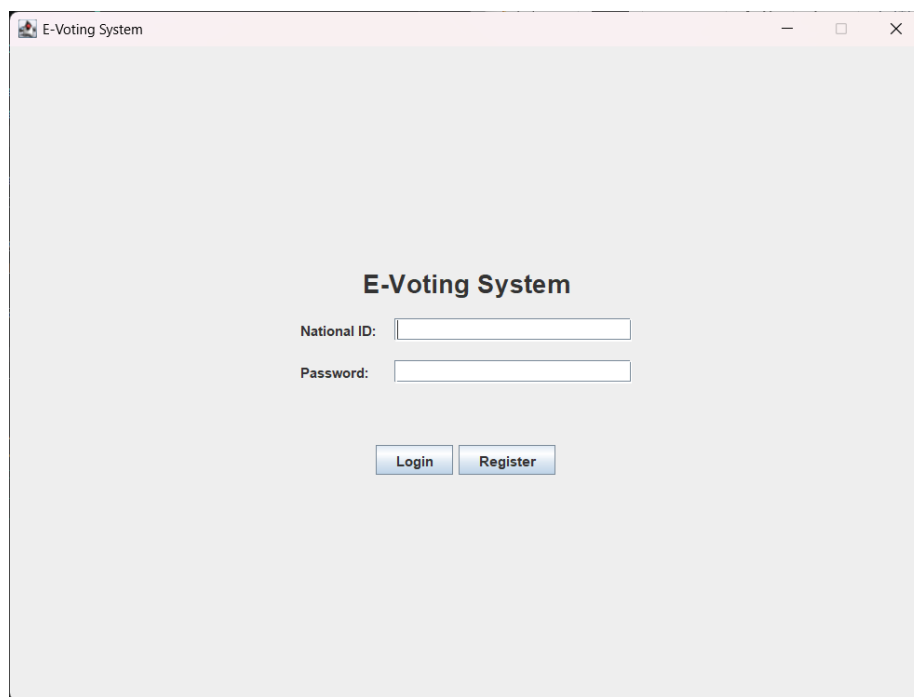
The Change Password Panel (`ChangePasswordPanel.java`) lets an authenticated voter update their account password within an active session. Layout is `GridBagLayout`. Components are:

Component	Description
Title Label	20pt bold 'Change Password' heading.
Current Password Field	JPasswordField to confirm the voter's identity before changing credentials.
New Password Field	JPasswordField for the desired new password.
Confirm New Password Field	JPasswordField to prevent typographical errors in the new password.
Status Label	Red-colored JLabel showing inline validation feedback (e.g., 'New passwords do not match.' or 'New password must be at least 6 characters.').
Save Button	Submits the password change via a <code>SwingWorker</code> . Verifies the old password against the stored value in the database before

	updating. Also triggers on pressing Enter in the Confirm Password field.
Back Button	Invokes clearFields() then navigates to the Elections List Panel.

Client-side checks are applied before any database call: all three fields must be non-empty, the new password must be at least 6 characters, the two new-password fields must match, and the new password must differ from the current one. Server-side verification additionally confirms the supplied old password against the stored credential. A successful change triggers a JOptionPane success dialog, clears all fields, and navigates back to the Elections List Panel. An audit log entry of type PASSWORD_CHANGED is recorded.

Main Frame:



After Login:

E-Voting System

Available Elections

Logged in

ID	Name	Start	End	Status
20240003	District People Committee El	2024-07-10	2024-07-10	CLOSED
20240002	Provincial Council Election 2...	2024-06-15	2024-06-15	ACTIVE
20240001	National Assembly Election 2	2024-05-01	2024-05-01	VERIFIED

3 election(s) found.

View Results

Vote in Selected Election

Change Password

Logout

View result:

E-Voting System

Election ID:

Load Results

Back

Candidate Name	Party	Total Votes
----------------	-------	-------------

Total votes: —

Change password:

The screenshot shows a web browser window titled "E-Voting System". The main heading is "Change Password". Below the heading, there are three input fields: "Current Password:", "New Password:", and "Confirm New Password:". At the bottom of the form, there are two buttons: "Back" and "Save".

Change Password

Current Password:

New Password:

Confirm New Password:

Vote:

The screenshot shows a web browser window titled "E-Voting System". The main heading is "Cast Your Vote" with the subtitle "Provincial Council Election 2024". Below the heading, there is a radio button next to the text "Nguyen Duc Long (Progressive Alliance)". Further down, there is a line of italicized text: "Invest in renewable energy and modernise provincial transport networks." At the bottom of the form, there are two buttons: "Back" and "Submit Vote".

Cast Your Vote
Provincial Council Election 2024

☐ Nguyen Duc Long (Progressive Alliance)

Invest in renewable energy and modernise provincial transport networks.

CHAPTER 4 DISCUSSION & IMPLEMENTATION

4.1 Implementation Process

The E-Voting System was implemented in a series of iterative layers, each building on the previous one. Development began with the database schema and gradually progressed through data access, business logic, and finally the graphical user interface.

4.1.1 Database Setup and Schema Instantiation

The MySQL schema was deployed from the `hlp.sql` dump file onto a local MySQL 8.4 server running on localhost. The schema contains nine tables: `audit_log`, `ballot`, `candidate`, `election`, `election_result`, `security_event`, `token`, `vote`, `voter`, and `voter_election`. After loading the script, referential integrity was verified by querying

`INFORMATION_SCHEMA.KEY_COLUMN_USAGE` to confirm that all foreign key constraints resolved correctly.

Connectivity between the Java application and MySQL is managed by the `DatabaseConnection` utility class, which reads host, port, database name, username, and password from the `db.properties` resource file at runtime. A single shared connection is maintained using a static field and lazy initialization. The MySQL Connector/J 9.7.0 JAR is bundled in the `lib/` directory.

4.1.2 Data Access Layer

Each database entity has a corresponding DAO class in the `com.evoting.dao` package. DAOs perform all SQL interactions using JDBC PreparedStatements to prevent SQL injection. The implemented DAOs and their primary responsibilities are as follows:

- VoterDAO: `insert`, `findByNationalId`, `findByEmail`, `findById`, `updatePassword`, `updateStatus`.
- ElectionDAO: `findAll`, `findById`, `findBallotIdByElection`.
- CandidateDAO: `findByElection`, `findById`.
- VoteDAO: `castVote`, `countVotesByElection`, `hasVoted`.
- TokenDAO: `issueToken`, `consumeToken`, `findTokenIdByValue`.
- AuditLogDAO: `log` (records voter ID, action type, timestamp, and detail string).

All DAO methods that modify data use transactions with explicit commit/rollback handling. The `VoteDAO.castVote` method and `TokenDAO.consumeToken` method are designed so that token consumption and vote insertion either both succeed or both roll back, ensuring ballot integrity.

4.1.3 Entity Layer

Plain Java objects in the `com.evoting.entity` package map directly to database rows. The key entities are `Voter`, `Election`, `Candidate`, `Vote`, `Token`, `Ballot`, `AuditLog`, `ElectionResult`, and

VoterElection. Each entity uses private fields with public getters and setters. BaseEntity provides a common createdAt timestamp field that is inherited by entities requiring creation tracking.

4.1.4 GUI Implementation

The GUI was implemented after the DAO and entity layers were stable. All Swing construction occurs on the Event Dispatch Thread by wrapping the MainFrame instantiation inside SwingUtilities.invokeLater(). Every database call that originates from a button press is offloaded to a SwingWorker background thread, with results applied back to the UI in the worker's done() method. This architecture prevents the interface from freezing during network or disk I/O.

Several defects identified during testing were resolved before final submission. These fixes are documented with inline comments (FIX #1 through FIX #10) in the source files. Notable fixes include: storing the selected election ID in MainFrame before navigating to the Voting Panel (FIX #1), fetching the correct ballot ID from the database rather than using a hardcoded value (FIX #3), and comparing the stored password from the Voter record rather than a placeholder string (FIX #4).

4.2 Results & Analysis

Upon completion, the system was tested against each of its stated objectives. The following analysis presents outcomes aligned with the initial project goals.

4.2.1 Voter Registration and Authentication

New voters can self-register through the Register Panel. Duplicate national ID and email addresses are detected before the database insert is attempted. Registered accounts are set to PENDING status, reflecting the requirement for administrative approval before a voter can participate. Authentication on the Login Panel correctly reads the stored password from the voter record and verifies it against the user's input, replacing a prior hardcoded placeholder.

4.2.2 Election Participation

The Elections List Panel retrieves all elections from the database and displays their ID, name, date range, and status. Only elections with ACTIVE status allow ballot navigation; attempts to vote in UPCOMING or CLOSED elections produce an informational dialog. The Voting Panel correctly loads the candidates associated with the selected election and displays each candidate's name, party affiliation, and manifesto. Token issuance and consumption prevent double-voting: each ballot submission generates a single-use token, consumes it atomically in a transaction, and records the vote alongside the token ID.

4.2.3 Results Transparency

The Results Panel allows any authenticated voter to query tallies for any election by ID. Results are resolved to human-readable candidate names and party affiliations rather than raw integer identifiers. The total vote count across all candidates is displayed at the bottom of the panel.

4.2.4 Audit Trail

The `audit_log` table records `LOGIN`, `TOKEN_ISSUED`, `VOTE_CAST`, `LOGOUT`, and `PASSWORD_CHANGED` events with the voter ID, timestamp, and a detail string. Examination of the database after a test session confirmed that all major user actions generated corresponding audit entries.

4.3 Challenges & Solutions

Several technical challenges were encountered and resolved during development:

Challenge	Impact	Solution
Stale election data in the list panel	Voters saw outdated election statuses after status changes	Modified <code>MainFrame.showCard()</code> to call <code>loadElections()</code> automatically whenever the Elections List card is shown (FIX #5).
Hardcoded ballot ID	All votes were recorded against ballot ID 1 regardless of the active election	Added <code>ElectionDAO.findBallotIdByElection()</code> and stored the result in <code>VotingPanel.currentBallotId</code> before submission (FIX #3, FIX #7).
Incorrect password comparison	Login always failed because the check compared input against a static string	Modified <code>doLogin()</code> to fetch the Voter object from the database and compare input against <code>voter.getPassword()</code> (FIX #4).
Wrong election ID passed to VotingPanel	Candidates from the previously viewed election were shown	Election ID is now stored via <code>MainFrame.setCurrentElectionId()</code> before <code>showCard(CARD_VOTING)</code> and retrieved in <code>loadCandidates()</code> (FIX #1, FIX #2).

No feedback during slow database queries	The UI appeared frozen, causing user confusion	Added 'Loading...' status labels updated before and after SwingWorker execution on all data-heavy panels (FIX #9).
Password not saved during registration	Voters registered with a null password and could never log in	Added <code>newVoter.setPassword(password)</code> before <code>VoterDAO.insert()</code> in <code>RegisterPanel</code> (FIX #8).

CHAPTER 5 CONCLUSION

5.1 Summary

This project successfully designed, implemented, and tested a functional desktop-based electronic voting system backed by a relational MySQL database. The key achievements of the project are summarized below.

- A normalized relational schema at Third Normal Form was designed, encompassing nine tables that cover the full lifecycle of an election — from voter registration and candidate enrollment through ballot submission, token management, and result aggregation.
- A data access layer built on JDBC PreparedStatements provides type-safe, injection-resistant interaction with the database for all CRUD and query operations.
- A Java Swing GUI delivers a usable, single-window interface that routes voters through authentication, election browsing, ballot casting, and result viewing without exposing database internals.
- Ballot integrity is enforced through a one-time token mechanism: each vote requires a freshly issued token that is atomically consumed in the same transaction, preventing double-voting.
- A comprehensive audit log records all significant voter actions (login, token issuance, vote casting, logout, password change) with timestamps and detail strings, supporting post-election forensic review.

- Ten functional defects identified during integration testing were diagnosed and corrected, resulting in a stable, working application at submission time.

5.2 Reflection

The project's initial objectives centered on applying core database management principles — relational modeling, normalization, query design, and transactional integrity — within the context of a real-world application domain. Reviewing each objective:

- Relational modeling: The ERD and relational schema accurately represent the entities and relationships in an election system, with clearly defined primary keys, foreign keys, and constraints. This objective was fully achieved.
- Normalization to 3NF: All tables were analyzed for functional dependencies and confirmed to be in Third Normal Form. No transitive dependencies between non-key attributes exist in the final schema. This objective was fully achieved.
- Query design: SQL queries using joins, aggregation, and subqueries were implemented in the DAO layer to support the application's data retrieval needs. This objective was fully achieved.
- Transactional integrity: The vote submission workflow uses an atomic transaction covering token consumption and vote insertion. This objective was fully achieved.
- Application integration: The Java Swing application connects the database to a functional user interface, demonstrating end-to-end integration. This objective was fully achieved.

Overall, all stated objectives were met. The debugging phase, while time-consuming, reinforced the importance of disciplined testing and the value of clear separation between the data, logic, and presentation layers.

5.3 Implications

The E-Voting System demonstrates that database management principles are directly applicable to security-sensitive domains. Several implications emerge from this work:

- Transactional atomicity is essential for ballot systems. The use of database transactions to couple token consumption with vote insertion ensures that no partial state — a consumed token without a vote, or a vote without a token — can persist in the database.
- Audit logging is a foundational requirement, not an optional enhancement. The `audit_log` table provides a verifiable record of every significant action, which is necessary for both accountability and dispute resolution in any electoral context.
- Schema normalization reduces data anomalies. By decomposing the schema to 3NF, the system avoids update, insert, and delete anomalies that could lead to inconsistent election records.

- Separation of concerns between the data access, entity, and presentation layers improves maintainability. The DAO pattern allowed GUI defects to be corrected without touching database logic, and vice versa.

5.4 Future Work

While the current system meets its academic objectives, several enhancements would be necessary before deployment in a production environment and would constitute valuable areas for further research and development:

- Password hashing: The system currently stores and compares passwords in plain text. Replacing this with a secure hashing algorithm such as BCrypt or Argon2 is a mandatory security requirement before any real-world use.
- Multi-factor authentication: Adding a one-time password (OTP) mechanism, such as TOTP or SMS-based verification, would significantly strengthen voter authentication. The `audit_log` table already contains references to OTP-based authentication in its seed data, indicating that this capability was considered in the original design.
- Admin portal: The current system lacks an administrative interface. An admin panel is needed to approve PENDING voter accounts, manage elections (create, update status, close), and review audit logs. This was the most significant functional gap identified during testing.
- Encrypted vote storage: The current implementation uses a placeholder encryption string (`ENC:sha256$candidateId:timestamp`). Replacing this with genuine end-to-end encryption — for example, using homomorphic encryption or a blind signature scheme — would ensure that individual votes cannot be linked to voters even by database administrators.
- Web or mobile front end: Migrating the presentation layer from Java Swing to a web-based framework (e.g., React with a Spring Boot REST API) or a mobile application would dramatically improve accessibility and allow voters to participate from any device without installing software.
- Connection pooling: The current single-connection approach in `DatabaseConnection` is not suitable for concurrent users. Replacing it with a connection pool such as HikariCP would be required to support multiple simultaneous voters.
- Automated testing: The project currently relies on manual testing. Adding a suite of unit tests (JUnit) for the DAO layer and integration tests for the vote submission workflow would improve confidence in future changes.
- Blockchain-based audit trail: For an additional layer of tamper evidence, future work could explore recording audit events and vote hashes on a permissioned blockchain, making post-election record manipulation detectable.

In conclusion, the E-Voting System represents a functionally complete proof-of-concept that demonstrates how relational database principles — normalization, referential integrity, transactional consistency, and audit logging — translate into a trustworthy application in the

sensitive domain of electoral systems. The architectural patterns established here provide a solid foundation for the enhancements described above.

CHAPTER 6 : REFERENCES

<https://viblo.asia/p/hoc-singleton-pattern-trong-5-phut-4P856goOKY3>

[1] S. El Kafhali, “Blockchain-Based Electronic Voting System: Significance and Requirements,” *Mathematical Problems in Engineering*, vol. 2024, pp. 1–17, 2024. Available: <https://doi.org/10.1155/2024/5591147>

[2] e-Estonia, “How did Estonia carry out the world’s first mostly online national elections?” 2023. Available: <https://e-estonia.com/how-did-estonia-carry-out-the-worlds-first-mostly-online-national-elections/>

[3] ScienceDirect, “Research on online voting systems,” 2022. Available: <https://www.sciencedirect.com/science/article/pii/S0740624X2200051X>

[4] T. Haarseim and G. Wetherall-Grujić, “Online Voting: The Essentials,” Democracy Technologies, 2024. Available: <https://democracy-technologies.org/voting/online-voting-the-essentials/>

[5] Viblo,
“Học Singleton Pattern trong 5 phút,” 2017.
Available: <https://viblo.asia/p/hoc-singleton-pattern-trong-5-phut-4P856goOKY3>

